



Formación

Curso Angular y WS REST

indra



Formador

Ana Isabel Vegas



INGENIERA INFORMÁTICA con Master Máster Universitario en Gestión y Análisis de Grandes Volúmenes de Datos: Big Data, tiene la certificación PCEP en Lenguaje de Programación Python y la certificación JSE en JavaScript. Además de las certificaciones SCJP Sun Certified Programmer for the Java 2 Platform Standard Edition, SCWD Sun Certified Web Component Developer for J2EE 5, SCBCD Sun Certified Business Component Developer for J2EE 5, SCEA Sun Certified Enterprise Architect for J2EE 5.

Desarrolladora de Aplicaciones FULLSTACK, se dedica desde hace + de 20 años a la CONSULTORÍA y FORMACIÓN en tecnologías del área de DESARROLLO y PROGRAMACIÓN.



training@iconotc.com

❑ **Duración:** 30 horas

❑ **Modalidad:** On-line **(No está autorizado la grabación de las sesiones on-line, según Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales.)**

❑ **Fechas/Horario:**

- Días 27, 28, 29, 30 y 31 Octubre 2025
- Horario 9:00 – 15:00 hs.

❑ **Contenidos:**

- Angular introducción

TypeScript

JavaScript ES6

Promesas

Decoradores

Funciones de flecha

Tipado de datos

Instalación AngularCLI

npm

Entorno

VSCode

- Primeros pasos con Angular

Estructura de proyecto

Directivas estructurales (*ngFor, *ngIf)

- Profundizar en Angular

Crear aplicación básica SPA

- Creación de componentes y reutilización

Navbar, Footer, Sidebar

Rutas (@Input, @Output)

Pipes

Estilos CSS, Sass, *ngStyle

Interceptores

Peticiones HTTP

Crear un loading

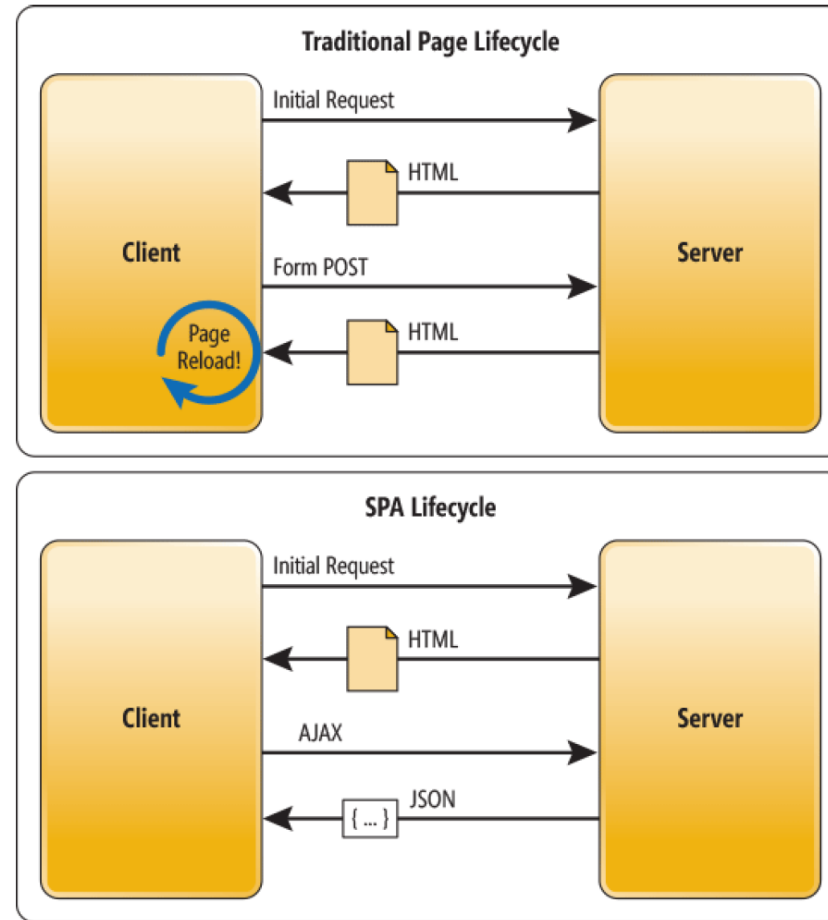
Internacionalización

Autenticación con Auth0

Qué es Angular?

- Básicamente es un Framework de JS que te permite crear Single Page Applications (o SPA's).
- Eso significa que con Angular nunca te va a abrir otra página. Todo será en la misma. Angular se encargará de ello. Esto lo hace de una forma muy elegante, simple y reactiva.

SPA



Versiones de Angular

- Angular 1: También conocido como AngularJS, versión inicial del Framework.
- Angular 2: Revisión completa del Framework. Se desecho AngularJS y se reescribió de nuevo. Esto porque AngularJS tenía errores. Literalmente era más sencillo crear un nuevo Framework que corregir el que ya se tenía.
- Angular 4: El primer update mayor de Angular 2.
- Angular 5: Cambios en el Core.
- Angular 6: Liberada en Mayo de 2018.

Versiones de Angular

- Angular 7: Octubre 2018.
- Angular 8: Octubre 2019.
- Angular 9: Febrero 2020.
- Angular 10: Junio 2020 y actualmente se conoce la ultima versión estable.

Versiones de Angular



AngularJS

1.0 , 1.1 , 1.2 .. 1.7



Angular

2.0 , 4.0 , 5.0 6.0,
7.0...



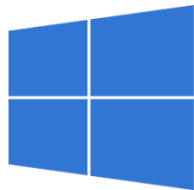
Puesta a punto del entorno

- Las herramientas que usaremos a lo largo del curso son:
 - **Navegador web** - Chrome es el que usaremos pero puedes usar el que te parezca mejor.
 - **Editor de texto** - Visual Studio Code, PHP Storm, Atom,, ...etc.
 - **Node.js** específicamente NPM.
 - **Angular CLI**, es un set de herramientas para la consola que nos facilita el trabajo con Angular.

Visual Studio Code

- Se puede descargar de esta url:

<https://code.visualstudio.com/download>



↓ Windows

Windows 7, 8, 10

User Installer	64 bit	32 bit
System Installer	64 bit	32 bit
.zip	64 bit	32 bit



↓ .deb

Debian, Ubuntu

↓ .rpm

Red Hat, Fedora, SUSE

.deb	64 bit	32 bit
.rpm	64 bit	32 bit
.tar.gz	64 bit	32 bit

[Snap Store](#)



↓ Mac

macOS 10.9+

Node.js

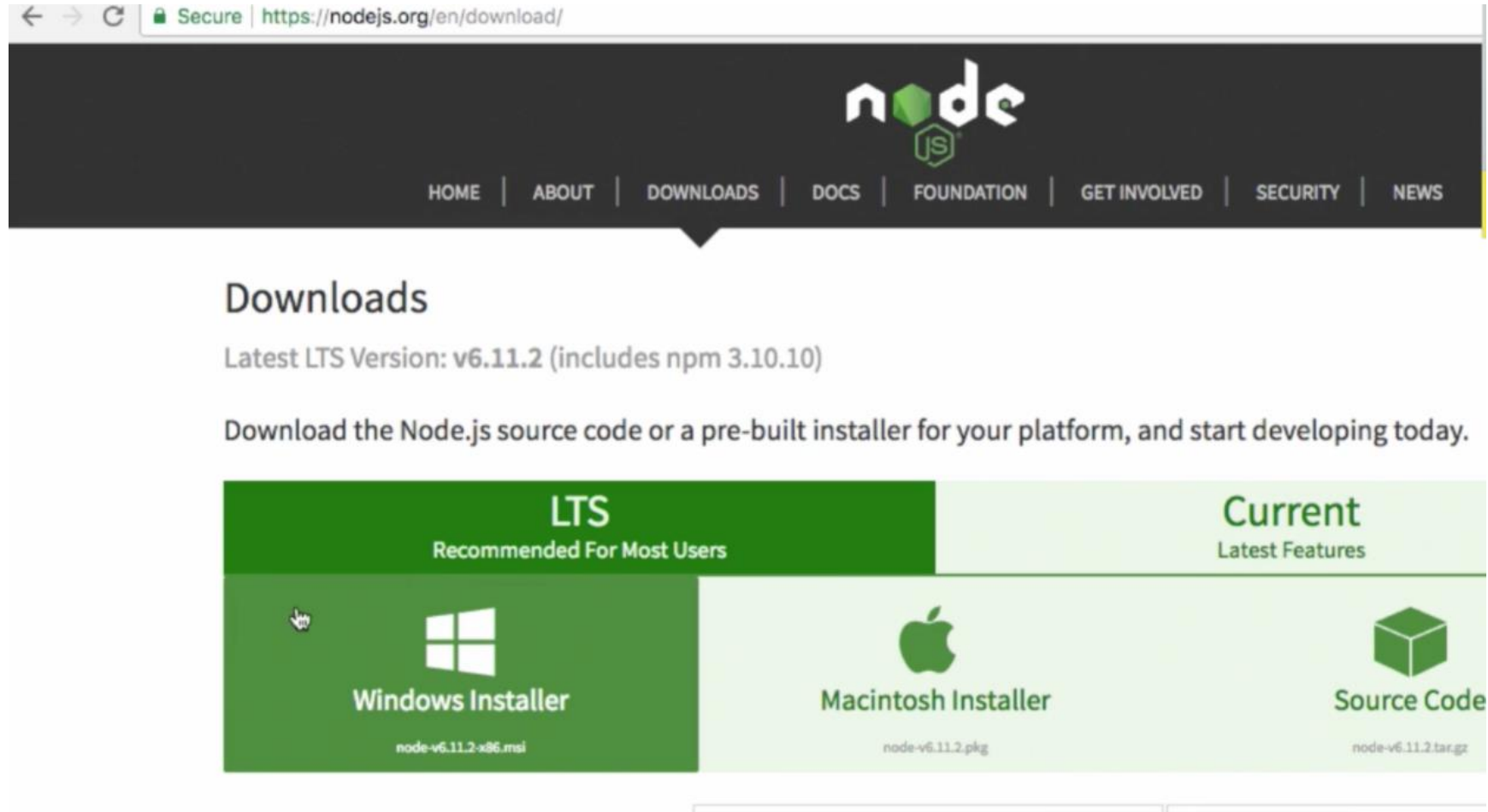
- Node.js, en la consola validamos que tenemos instalado npm con el comando:

```
npm -v
```

- De no ser así nos lo descargamos de la página oficial:

<https://nodejs.org/en/download/prebuilt-installer>

Node.js



Angular CLI

- Recuerda ejecutar los comandos con privilegios de administrador.

```
npm config set proxy http://10.0.8.102:8080
```

```
npm config rm proxy
```

```
npm install -g @angular/cli
```

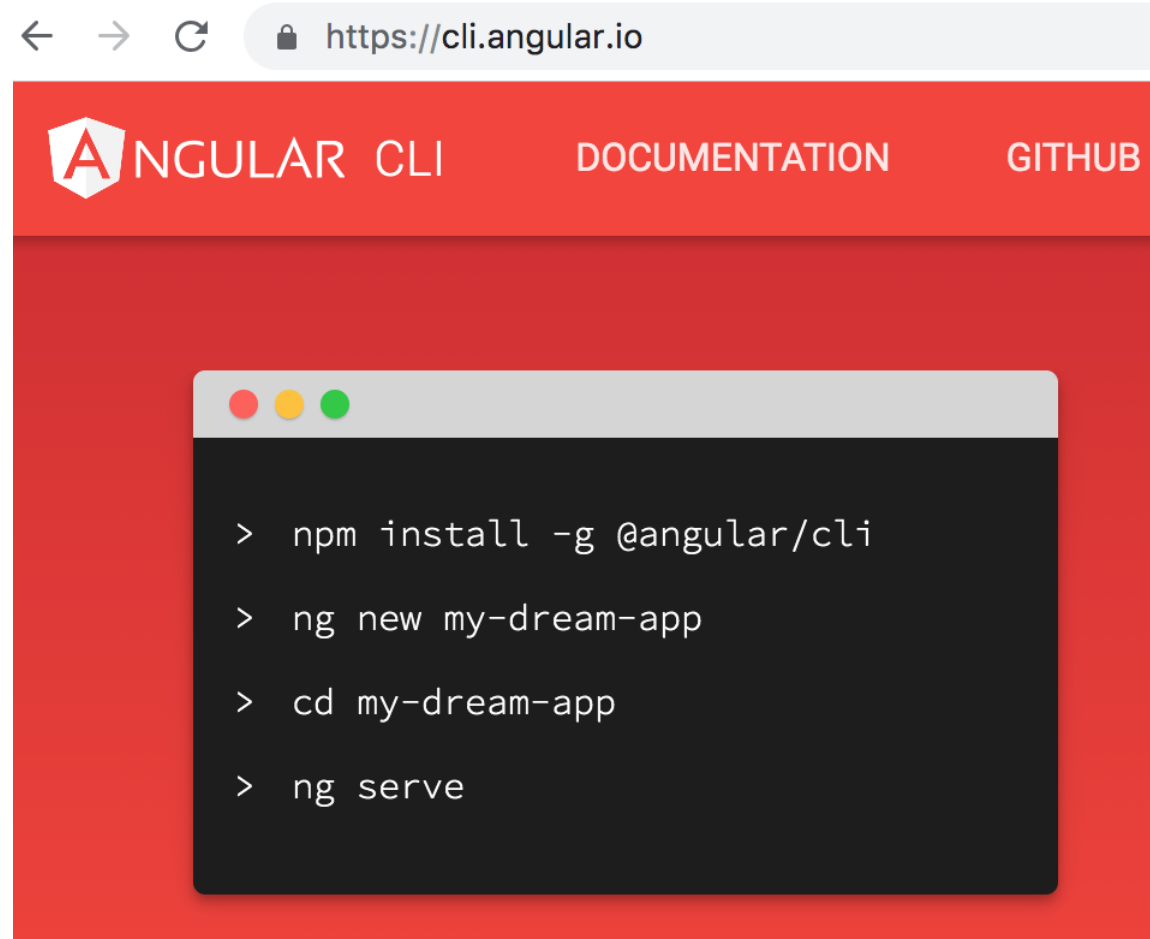
- Si habíamos instalado una versión de Angular-CLI previamente y queremos actualizarla, debemos ejecutar los siguientes comandos:

```
1. npm uninstall -g @angular/cli
```

```
2. npm cache clean --force
```

```
3. npm install -g @angular/cli@13.0.0
```

Nuestro primer proyecto



The image shows a file explorer window for a project named 'curso-angular4'. The file list includes folders like 'e2e', 'node_modules', and 'src', and various files such as 'app.component.css', 'app.component.html', 'app.component.spec.ts', 'app.component.ts', 'app.module.ts', 'assets', 'environments', 'favicon.ico', 'index.html', 'main.ts', 'polyfills.ts', 'styles.css', 'test.ts', 'tsconfig.app.json', 'tsconfig.spec.json', 'typings.d.ts', '.angular-cli.json', '.editorconfig', '.gitignore', 'curso-angular4.sublime-project', 'curso-angular4.sublime-workspace', 'karma.conf.js', 'package.json', 'protractor.conf.js', 'README.md', 'tsconfig.json', and 'tslint.json'.

Archivos estáticos (imágenes de la aplicación)

Estilos globales de la aplicación

- **.ts**: type script
- **app.component.ts**
Componente principal de la aplicación donde se define la clase del componente (AppComponent):
- **.html** es la plantilla del componente
- **.css** especifica la presentación/vista del componente

Podemos aplicar MVC para la estructuración de las carpetas del proyecto Angular.

package.json: el archivo de configuración de un proyecto de Node. Contiene las dependencias/librerías/paquetes de Angular que vamos a utilizar en nuestra aplicación y que Angular CLI (via NPM) se ha encargado de instalarlas por nosotros en la carpeta `node_modules`. Estas son las librerías imprescindibles para nuestro proyecto y podremos seguir añadiendo nuevas librerías para cubrir las funcionalidades y requisitos del proyecto.

index.html: es dónde se va a *renderizar* la aplicación

Eliminar node-modules

- Si el proyecto pesa demasiado podemos eliminar la carpeta node-modules.
- Nos dara errores de compilacion y no podremos ejecutar el proyecto.
- Para volver a descargar todas las dependencias:
 1. Nos posicionamos dentro del proyecto en la consola
 2. Ejecutamos: `npm install`

Módulos en Angular

- Los módulos son las piezas mas grandes en las que se puede dividir una aplicación Angular.
- Son piezas que podemos reutilizar dentro de nuestro proyecto pero también podemos usarlas en otros proyectos.
- Angular nos ofrece módulos nativos como HTTP, Browser, Forms, Reactive

Módulos en Angular

- Categorías de módulos:
 1. Nativos: provistos por Angular.
 2. De terceros: realizados por otras personas y que nosotros podemos usar, siendo libres o pago.
 3. Propios: los que realicemos nosotros.
- Para anotar nuestros módulos

`src/app/app.module.ts`

Módulos en Angular

- @NgModule: Palabra reservada para declarar un Modulo, contiene 4 arrays:
 - declarations: Lista de componentes
 - imports: Lista de modulos
 - providers: Lista de servicios
 - bootstrap: Componente de inicio

Módulos en Angular

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Componentes en Angular

- Un módulo en un app, tiene una entidad propia y puede estar compuesto por más cosas como componentes.
- Cada vista debería ser un componente, y está puede o no, componerse por otros componentes, dependiendo de la complejidad, profundidad y la reutilización de código.

Componentes en Angular

- Categorías:
 - Imports: módulos externos a importar.
 - Providers: servicios necesarios.
 - Bootstrap: con que componente vamos a iniciar.
- @Component: Palabra reservada para declarar un componente, contiene varios elementos:
 - selector: Tag de HTML para insertar el componente
 - templateUrl: Archivo HTML que va a usar el componente
 - styleUrls: Archivos CSS que usará el template

Componentes en Angular

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})
```

Data Binding

- Data binding es la comunicación entre tu código de TypeScript y el HTML. Siempre ten presente que al cliente le importa lo que ve, así que comunicar nuestra lógica (TypeScript) al template (HTML) es muy importante, para esto contamos con 4 tipos de Data Binding,

Tipos de Data Binding en Angular

- **String Interpolation** `{{}}` TypeScript $\Rightarrow$ HTML: Tener información (Variable, Array, por ejemplo) y presentarla a los usuarios en modo de HTML.
- **Property Binding** `[]` TypeScript $\leq$ HTML: Información del lado de HTML que puede ser por ejemplo, información que el usuario ingrese o que nosotros pongamos un valor por defecto. Viajando la información de HTML a TypeScript.
- **Event Binding** `()` TypeScript $\leq$ HTML: Escuchar eventos desde HTML y pasarlo a TypeScript.
- **Two Way Data Binding** `[()]` TypeScript $\Leftrightarrow$ HTML: Comunicación de dos Vías. De lo que el cliente ve a TypeScript como de TypeScript hacia lo que el cliente ve.

Comandos ng

Scaffold	Usage
Component	<code>ng g component my-new-component</code>
Directive	<code>ng g directive my-new-directive</code>
Pipe	<code>ng g pipe my-new-pipe</code>
Service	<code>ng g service my-new-service</code>
Class	<code>ng g class my-new-class</code>
Guard	<code>ng g guard my-new-guard</code>
Interface	<code>ng g interface my-new-interface</code>
Enum	<code>ng g enum my-new-enum</code>
Module	<code>ng g module my-module</code>

Directivas

- Las directivas son una forma elegante y rápida de manipular la información, contamos con 3 tipos de directivas:
 - COMPONENTES: Son directivas que siempre tienen asignados templates de HTML.
 - ESTRUCTURALES: Nos permiten modificar el DOM, es decir manipular elementos de HTML.
 - ATRIBUTOS: A estas directivas les indicamos a través de HTML cómo se deben comportar de acuerdo con ciertas condiciones que definimos.

Directiva ngFor

- Es una directiva estructural. Nos permite iterar sobre una colección.

```
<ul>  
  <li *ngFor="let hero of heroes">{{hero.name}}</li>  
</ul>
```

Directiva ngIf

- Directiva ngIf es una directiva estructural, ésta directiva nos permite mostrar elementos de HTML de acuerdo a una condición que definamos.

```
<div *ngIf="hero" class="name">{{hero.name}}</div>
```

Directiva ngStyle

- Es una directiva de atributo por eso va entre corchetes.
- ngStyle se utiliza para aplicar un estilo CSS en función de una condición:

```
<some-element [ngStyle]="{'font-style': styleExp}>...</some-element>
```

```
<some-element [ngStyle]="{'max-width.px': widthExp}>...</some-element>
```

```
<some-element [ngStyle]="objExp">...</some-element>
```

Directiva ngClass

- Es otra directiva de atributo.
- ngClass se utiliza para aplicar una clase de estilo definida en un fichero css.

```
<some-element [ngClass]="['first second']">...</some-element>
```

```
<some-element [ngClass]="['first', 'second']">...</some-element>
```

```
<some-element [ngClass]="{'first': true, 'second': true, 'third': false}">...</some-element>
```

```
<some-element [ngClass]="stringExp|arrayExp|objExp">...</some-element>
```

```
<some-element [ngClass]="{'class1 class2 class3' : true}">...</some-element>
```

Directivas ngSwitch / ngSwitchCase

- ngSwitch -> directiva de atributo
- ngSwitchCase -> directiva estructural

```
<div [ngSwitch]="hero?.emotion">
  <app-happy-hero    *ngSwitchCase="'happy'"    [hero]="hero"></app-happy-hero>
  <app-sad-hero      *ngSwitchCase="'sad'"      [hero]="hero"></app-sad-hero>
  <app-confused-hero *ngSwitchCase="'app-confused'" [hero]="hero"></app-confused-hero>
  <app-unknown-hero  *ngSwitchDefault          [hero]="hero"></app-unknown-hero>
</div>
```


Directivas personalizadas

- Podemos utilizar las directivas propias de Angular o crear las nuestras propias.
- Una buena practica es crear una carpeta directivas para tal fin.
- En dicha carpeta creamos un archivo typescript donde definimos la directiva.

Directivas personalizadas

```
@Directive({
```

```
    Selector: nombre a utilizar en el html
```

```
})
```

```
Export class{
```

```
    // implementacion de la directiva
```

```
}
```

HostListener

- Los *host listeners* nos ayudan a escuchar los eventos de manera muy sencilla, tiene tres parámetros, el evento, el objetivo del evento y el código a ejecutar cuando se ejecute el evento.

HostBinders

- Host Binding nos permite editar elementos del DOM o el HTML al que esté asignado desde nuestra directiva.
- Recibe el atributo del HTML que queremos modificar.

Pipes

- Los pipes son elementos que usamos en el DOM en el HTML junto con las directivas que van a tomar un elemento de entrada y le van a dar cierto formato y nos van a entregar una salida diferente.
- Angular trae por defecto una cantidad de pipes para configuraciones y cambios comunes. Por ejemplo, cambiar las letras de mayúsculas a minúsculas, formatear fechas, etc.
- Los pipes pueden tomar parámetros que le indiquemos.
- Es posible que encadenemos pipes hasta que obtengamos el resultado que deseamos.
- Además de los pipes que Angular tiene por defecto, es posible que nosotros hagamos nuevos.

Ejemplos

- `{{ texto | uppercase }}`
- `{{ fecha | date:'short' }}`
- `{{ importe | currency:'€ ':'code':'.1-1' }}`

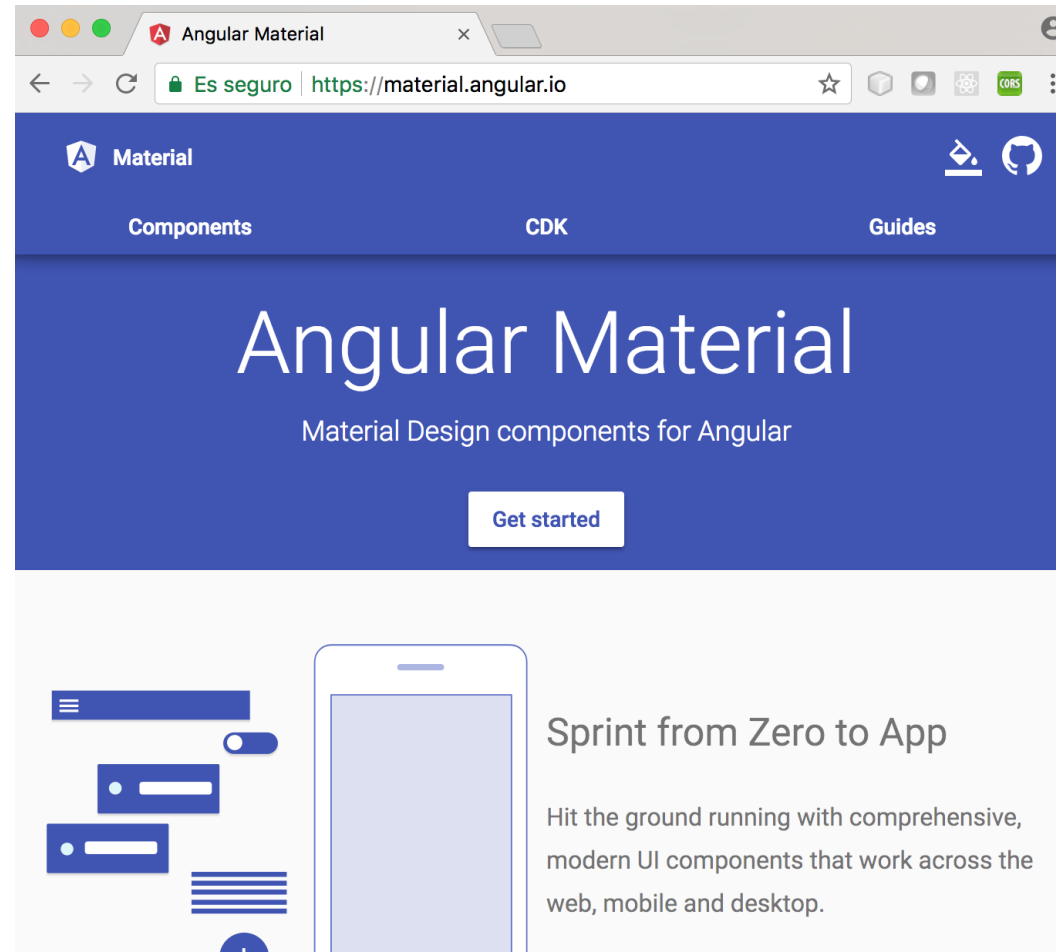
```
<!-- npm i @angular/localize
en angular.json incluir i18n con locale es

"projects": {
  "Ejemplo6-pipes": {
    "i18n":{
      "sourceLocale": "es"
    },
    "projectType": "application",
  }
}
-->
```

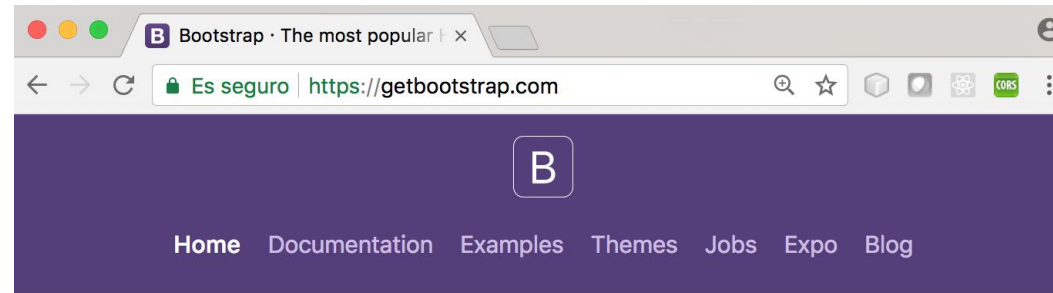
Angular UI

- Angular por defecto no posee estilos, pero se pueden agregar de forma muy sencilla, la parte visual de la aplicación es muy importante porque es lo que mas le interesa al usuario final.
- Angular Material es un framework de estilos para tus aplicaciones.
- Otra alternativa es bootstrap, es un framework para hacer interfaces que tiene estilos muy bonitos, y una muy buena guía, nos permite implementar el diseño responsive de forma muy sencilla y cómoda

Angular Material



Bootstrap



Bootstrap

Build responsive, mobile-first projects on the web with the

Bootstrap

- Instalación en el proyecto

```
$ npm install bootstrap
```

- Agregar bootstrap en angular.json

```
  "styles": [  
    "node_modules/bootstrap/dist/css/bootstrap.min.css",  
    "src/styles.css"  
  ],
```

Jquery y Popper

- Instalación en el proyecto

```
npm install jquery
```

```
npm i @popperjs/core
```

- Agregar jquery en angular.json

```
    "styles": [  
      "node_modules/bootstrap/dist/css/bootstrap.min.css",  
      "src/styles.css"  
    ],  
    "scripts": [  
      "node_modules/jquery/dist/jquery.js",  
      "node_modules/@popperjs/core/dist/umd/popper.min.js",  
      "node_modules/bootstrap/dist/js/bootstrap.min.js"  
    ]  
  }
```

Router

- El *routing* en Angular nos permite implementar navegación en nuestra aplicación, podemos asignar vistas/componentes específicos para cada url que deseemos. Routing nos permite enviar y recibir parámetros.

```
// Importar modulo Routes
import { Routes } from '@angular/router';

// Array para definir todas las rutas de navegacion
const misRutas: Routes = [
  {path:'conciertos', component: ConciertosComponent},
  {path:'detalle/:codigo', component:DetalleComponent},
  {path:'alta', component:AltaFormComponent},
  {path:'login' , component:LoginComponent}
];
```

Router

- Este es un ejemplo de cómo redireccionar a otros componentes:

```
const misRutas: Routes = [  
  { path: 'us', component: NosotrosComponent },  
  { path: 'contact', component: ContactoComponent },  
  { path: 'tab', component: TablaComponent },  
  { path: 'index', component: HomeComponent },  
  { path: '', redirectTo: 'index', pathMatch: 'full' }  
];
```

RouterModule

- Para poder agregar un nuevo componente que va a tener su propia ruta asignada necesito el modulo de RouterModule
- Importo RouterModule en app.module.ts:

```
// Importar modulo RouterModule  
import { Routes, RouterModule } from '@angular/router';
```

```
imports: [  
  BrowserModule,  
  RouterModule.forRoot(misRutas),
```

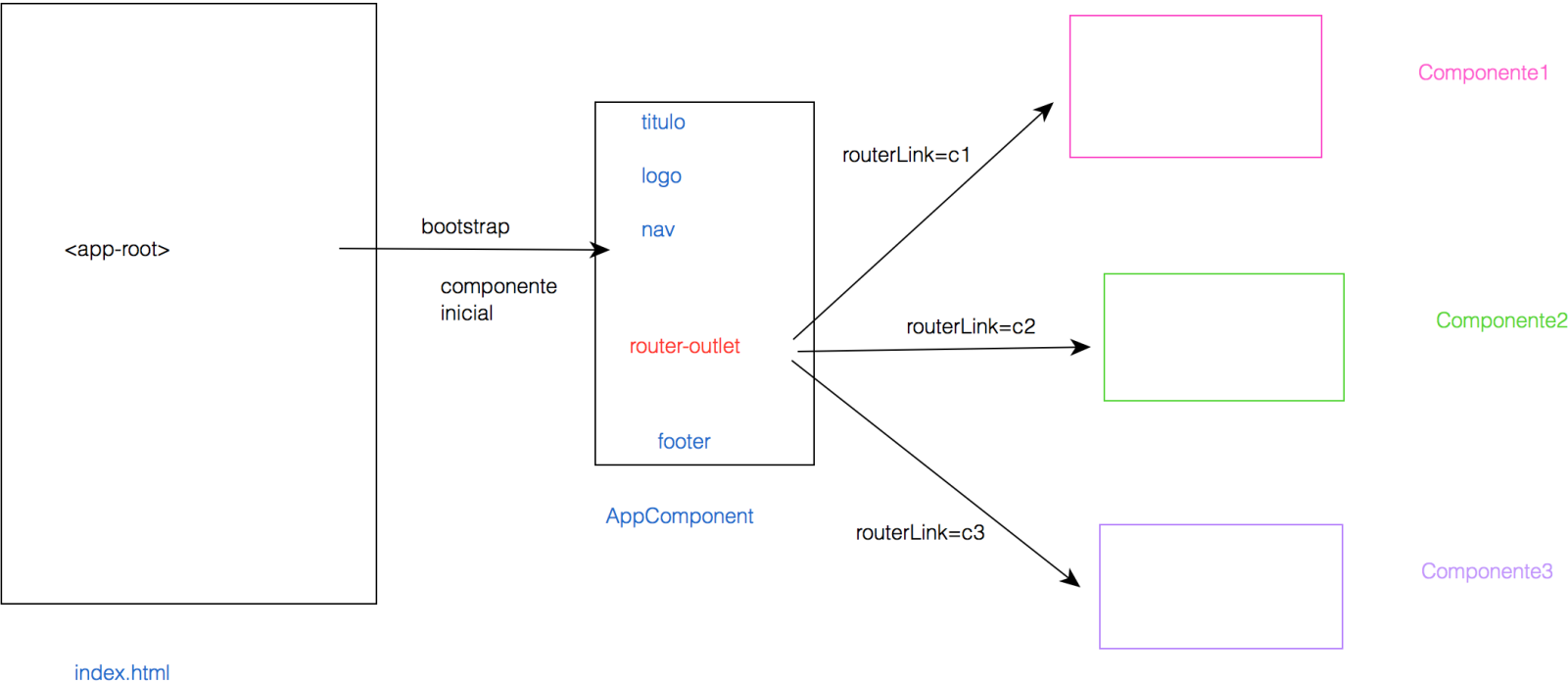
routerLink

- Para implementar navegación en SPA's la mejor forma de hacerlo es usando el *routerLink* que nos provee Angular ya que así reducimos los tiempos de carga y aprovechamos al máximo las capacidades del *framework*.

href y routerLink

- Angular es ideal para SPA (Single Page Application), la diferencia esta en:
 - **href** -> enlace tradicional, con redirección, si se hace click te lleva a otra página (como si fuera otra distinta), recargando todo, no recomendable para una SPA.
 - **routerLink** -> mantiene el index.html y usando `<router-outlet></router-outlet>` quita y pone el componente al que hace referencia, mostrándolo en esa parte del index.html.

Ejemplo Router



Link activo

- El router nos permite resaltar el link que se encuentra activo.
- Puedes usar `routerLinkActive="cualquier clase"`, le puede indicar a angular la clase que hace que se resalte el link que están visitando, para este caso bootstrap usa la clase `active`

```
<li class="nav-item">
  <a class="nav-link" routerLink="detalle/3"
    routerLinkActive="active">Detalle</a>
</li>
<li class="nav-item">
  <a class="nav-link" routerLink="contacto"
    routerLinkActive="active">Contacto</a>
</li>
```

Parámetros en Rutas

- Ruta/:parametro

```
const misRutas: Routes = [  
  {path: 'inicio', component: AppComponent},  
  {path: 'contacto', component: ContactoComponent},  
  {path: 'catalogo', component: CatalogoComponent},  
  {path: 'detalle/:id', component: DetalleComponent}  
];
```

Parámetros tipo Query

- Son parámetros opcionales que declaro en el enlace con la directiva queryParams.

```
<li class="nav-item">
  <a class="nav-link" routerLink="catalogo"
    [queryParams]= "{categoria:'zapatos',color:'negro'}"
    routerLinkActive="active">Catalogo</a>
</li>
```

Parámetros en Rutas

- Para recuperar los parámetros:

```
constructor(private route: ActivatedRoute) {  
  this.codigo = this.route.snapshot.params['id'];  
}
```

```
constructor(private route: ActivatedRoute) {  
  this.categoria = this.route.snapshot.queryParams['categoria'];  
  this.color = this.route.snapshot.queryParams['color'];  
}
```

Servicios

- Los servicios nos permiten centralizar funcionalidad de nuestra aplicación, pueden ser usados desde cualquier componente y hacen nuestra aplicación mas mantenible.

Crear un servicio

- Desde linea de comandos podemos crear el servicio.
- Es importante poner el nombre de la carpeta para que no lo ubique en app.

```
[MacBook-Pro-de-Ana:MyGoogleMaps anaisabelvegascaceres$ ng g service services/negocios  
CREATE src/app/services/negocios.service.spec.ts (343 bytes)  
CREATE src/app/services/negocios.service.ts (137 bytes)
```

Declaración del servicio

```
import { Injectable } from '@angular/core';
import { Negocio } from '../modelos/Negocio';

@Injectable({
  providedIn: 'root'
})
export class NegociosService {

  negocios: Negocio[] = [
    {id:1,nombre:'Bar Numancia',lat:40.4359819,
    {id:2,nombre:'Rodilla',lat:40.4335952 ,long
    {id:3,nombre:'Grupo Atrium',lat:40.437171 ,
    {id:4,nombre:'KutxaBank',lat:40.4358394 ,lo
    {id:5,nombre:'AulaCenter',lat:40.4367342 ,l
    {id:6,nombre:'Cafeteria Albahaca',lat:40.43
  ];

  public getNegocios(){
    return this.negocios;
  }
}
```


Inyectar el servicio

- El componente que desee hacer uso del servicio debe inyectarlo

```
constructor(private negociosService: NegociosService) {  
    this.negocios = negociosService.getNegocios();  
}
```

Http

- Importar el modulo:

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
```

Http

- Declarar variables a utilizar:

```
url: "http://176.655.54.12:8080/miAplicacion/servicioREST";  
bbdd = "https://gestioncroquetas.firebaseio.com"  
headers = new HttpHeaders({"Content-type":"application/json"});
```

Http

- Peticiones get y post:

```
constructor(private http:HttpClient) { }

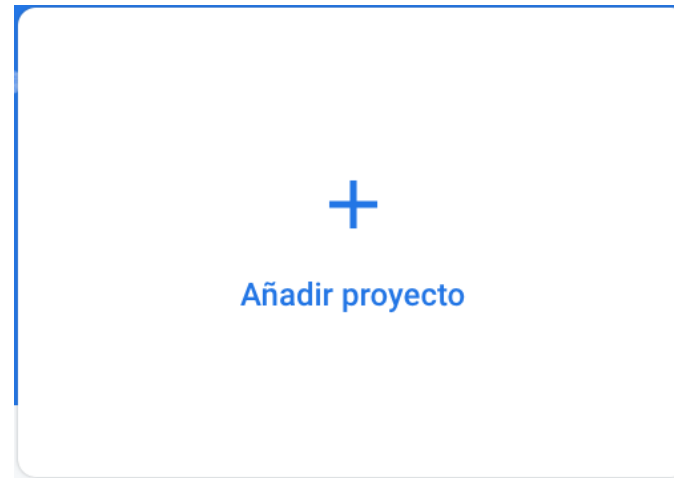
public insertarReceta(receta){
    return this.http.post(this.bbdd + "/recetas.json", receta, {headers: this.headers});
}

public getRecetas(){
    return this.http.get(this.bbdd + "/recetas.json");
}
```

Firestore

<https://firebase.google.com>

[Ir a la consola](#)



Firestore

- Una vez creado el proyecto en Firebase lo agregamos al proyecto web
- Nos mostrará los datos de configuración que pegaremos en el archivo environment.ts



Firestore

```
export const environment = {  
  production: false,  
  firebase: {  
    apiKey: "AIzaSyD5FKX0na1Lr5kGJWC19kPILIsIq_dIrsA",  
    authDomain: "anabel-indra-productos.firebaseio.com",  
    databaseURL: "https://anabel-indra-productos.firebaseio.com",  
    projectId: "anabel-indra-productos",  
    storageBucket: "anabel-indra-productos.appspot.com",  
    messagingSenderId: "855940992652",  
    appId: "1:855940992652:web:d5f6506f755bd2f30735a9",  
    measurementId: "G-HH7Z0PL83N"  
  }  
};
```

Firestore

- Instalamos en el proyecto el modulo de Firebase:

```
npm i @angular/fire
```

```
npm i firebase
```

```
"@angular/fire": "^7.6.1",
```


Firestore

```
import { AngularFireModule } from '@angular/fire/compat';  
import { AngularFireDatabaseModule } from '@angular/fire/compat/database';  
import { environment } from 'src/environments/environment';
```

```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    FormsModule,  
    ReactiveFormsModule,  
    AngularFireModule.initializeApp(environment.firebase),  
    AngularFireDatabaseModule  
  ],  
})
```

Firestore

```
import { Injectable } from '@angular/core';
import { AngularFireDatabase, AngularFireList } from '@angular/fire/compat/database';
import { Contacto } from '../models/contacto';

@Injectable({
  providedIn: 'root'
})
export class AgendaService {

  private db = '/agenda';
  agendaRef: AngularFireList<Contacto>;

  constructor(private angularDB: AngularFireDatabase) {
    this.agendaRef = this.angularDB.list(this.db);
  }

  getAll(): AngularFireList<Contacto> {
    return this.agendaRef;
  }
}
```

Firestore

- Inyectar el servicio en el componente para cargar la lista de contactos al iniciar:

```
constructor(private agendaService: AgendaService){  
  this.agendaService.getAll().snapshotChanges().pipe(  
    map(cambio => cambio.map(c => (  
      {key: c.payload.key, ...c.payload.val()}  
    )))  
  ).subscribe(datos => {  
    this.contactos = datos;  
  });  
}
```

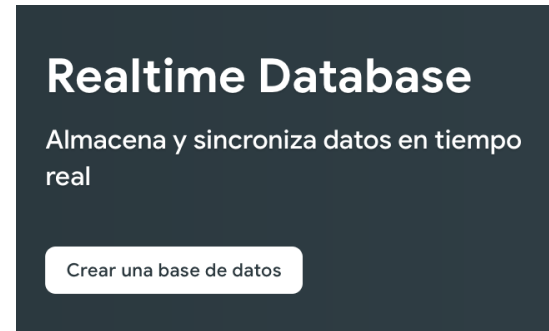
Firestore

- Crear la vista para que muestre todos los productos:

```
<h2>Todos los contactos</h2>
<ul>
  <li *ngFor="let contacto of contactos">
    | {{contacto.nombre}} {{contacto.telefono}}
    <button (click)="borrar(contacto.key)">Eliminar</button>
    <button (click)="modificar(contacto)">Modificar</button>
  </li>
</ul>
```

Firestore

- Solo falta crear la colección en Firebase.



El parámetro de configuración de la ubicación determina en qué lugar se almacenarán tus datos de Realtime Database.

Ubicación de Realtime Database

Firestore

Cuando defines la estructura de los datos, **deberás crear reglas para protegerlos.**

[Más información](#) 



Comenzar en **modo bloqueado**

Tus datos son privados de forma predeterminada. El acceso de lectura/escritura de los clientes solo se otorgará como se indica en tus reglas de seguridad.



Comenzar en **modo de prueba**

Para permitir una configuración rápida, los datos se abren de forma predeterminada. Sin embargo, debes actualizar las reglas de seguridad dentro de 30 días a fin de habilitar el acceso de lectura/escritura a largo plazo para los clientes.

```
{
  "rules": {
    ".read": "now < 1700953200000", // 2023-11-26
    ".write": "now < 1700953200000", // 2023-11-26
  }
}
```



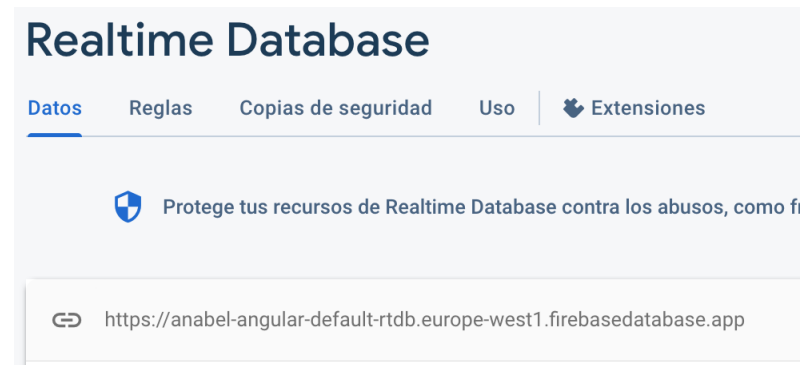
Las reglas de seguridad predeterminadas del modo de prueba permiten que cualquier usuario con acceso a tu referencia de base de datos pueda ver, editar y borrar todos los datos durante los siguientes 30 días.

Cancelar

Habilitar

Firestore

- Agrego la url de la base de datos a la configuración del environment:



```
export const environment = {  
  production: false,  
  firebase: {  
    apiKey: "AIzaSyCqJlREG0rSTc59I9FZtU4wSza49iitSmg",  
    authDomain: "anabel-angular.firebaseio.com",  
    databaseURL: "https://anabel-angular-default-rtdb.europe-west1.firebaseio.com",  
    projectId: "anabel-angular",  
    storageBucket: "anabel-angular.appspot.com",  
    messagingSenderId: "255860274083",  
    appId: "1:255860274083:web:0fa8630af5082ee3976016"  
  }  
};
```

Formularios Reactivos

- Hay que importar los modulos en app.module.ts

```
import { FormsModule, ReactiveFormsModule } from '@angular/forms';  
  
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    AngularFireModule.initializeApp(environment.firebase),  
    FormsModule,  
    ReactiveFormsModule  
  ]  
})  
export class AppModule {}
```


Formularios Reactivos

```
export class AppComponent implements OnInit{
  public catalogo = [];
  public nuevoProductoForm = new FormGroup({
    nombre: new FormControl('', Validators.required),
    precio: new FormControl('', Validators.required)
  });

  altaProducto(){
    let data = {
      nombre: this.nuevoProductoForm.value.nombre,
      precio: this.nuevoProductoForm.value.precio
    }
    this.crudService.insertarProducto(data).then(() => {
      console.log("Producto insertado");
      this.nuevoProductoForm.setValue({
        nombre: '',
        precio: ''
      });
    }, (error) => {
      console.error(error);
    });
  }
}
```

Formularios Reactivos

```
<form [formGroup]='newProductoForm' (submit)='altaProducto()'>
  <label>Nombre:</label>
  <input type="text" formControlName="nombre" /> <br/>

  <label>Precio:</label>
  <input type="text" formControlName="precio" /> <br/>

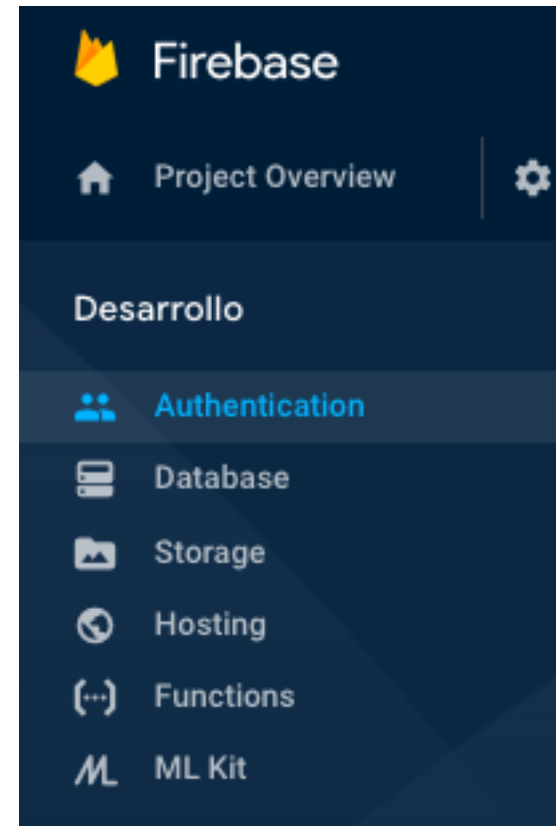
  <input type="submit" value="Agregar Producto"
    [disabled]='!newProductoForm.valid' />
</form>
```

Login

- Firebase nos da la opción de poder gestionar los usuarios de nuestra aplicación y poder resolver la autenticación.
- Es necesario tener un proyecto creado en Firebase y los datos de acceso en el archivo `environment.ts`

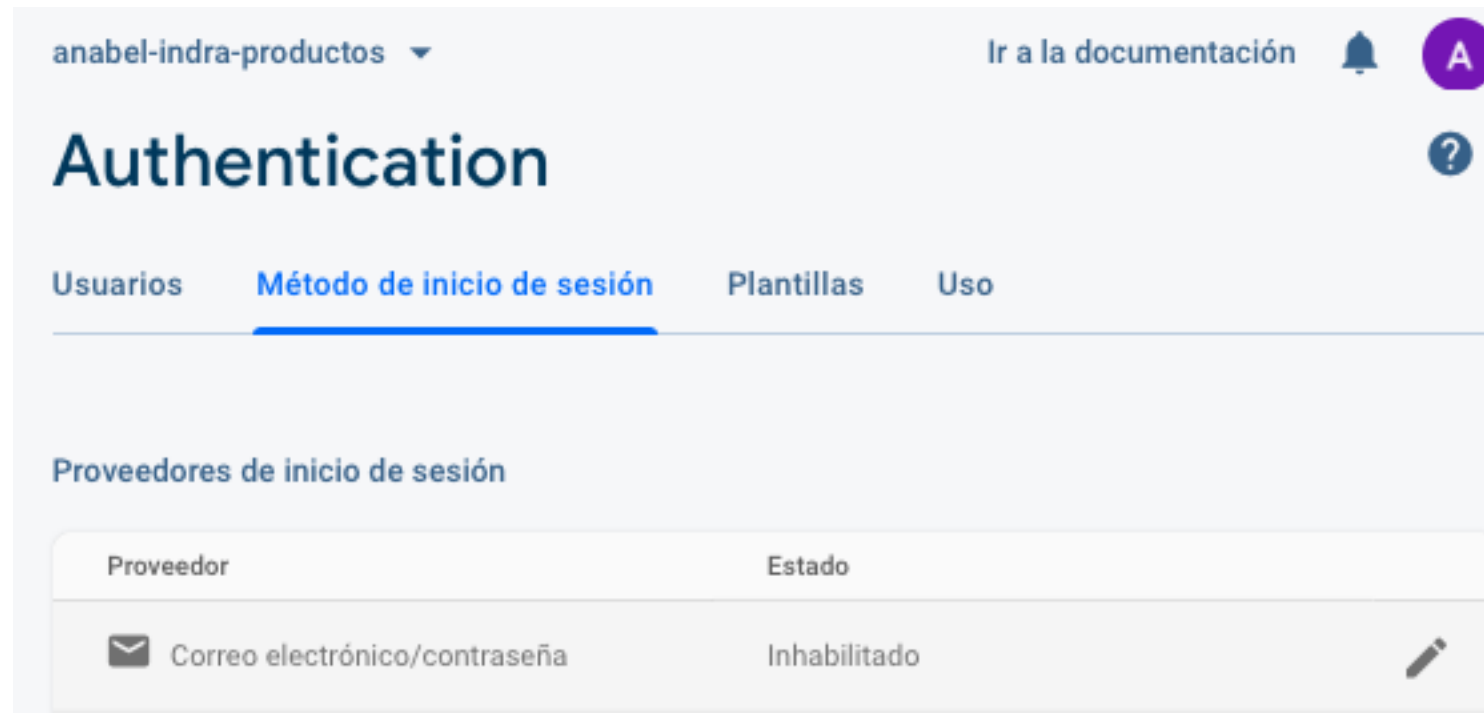
Login

- Desde la consola de Firebase, en nuestro proyecto...
- Pulsamos sobre la opción de Autenticación



Login

- Elegimos el modo de Inicio de sesión que será a través de Correo electrónico y contraseña



Login

- Habilitar el primer check

 Correo electrónico/contraseña

☒

Habilitar

Permite a los usuarios registrarse con su dirección de correo electrónico y contraseña. Nuestros SDK también proporcionan la verificación de la dirección de correo, la recuperación de contraseñas y las primitivas de cambio de dirección de correo.

[Más información](#) 

Enviar enlace por correo electrónico (inicio de sesión sin contraseña)

☐

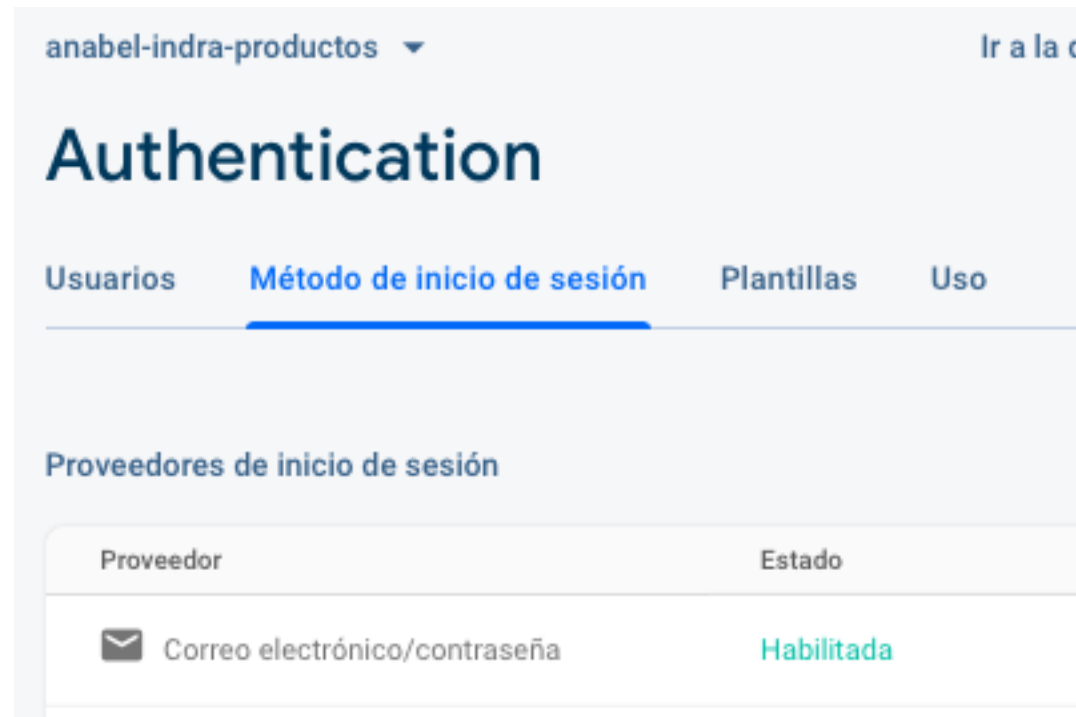
Habilitar

Cancelar

Guardar

Login

- Ya tenemos la opción habilitada



Login

- Una vez instaladas las librerías de Firebase en el proyecto, inyectamos el servicio de autenticación en el servicio.

```
import { Injectable } from '@angular/core';
import { AngularFireAuth } from '@angular/fire/auth';

@Injectable({
  providedIn: 'root'
})
export class LoginService {

  constructor(private angularFireAuth: AngularFireAuth) { }
}
```


Login

```
// Hacer login cuando esta registrado
loginWithEmail(email:string, password:string){
  return this.angularFireAuth.auth
    .signInWithEmailAndPassword(email,password);
}

// Si no esta registrado se puede registrar
register(email:string, password:string){
  return this.angularFireAuth.auth
    .createUserWithEmailAndPassword(email,password);
}

// logOut
logout(){
  return this.angularFireAuth.auth.signOut();
}

// Comprobar si esta logado
getStatus(){
  return this.angularFireAuth.authState; // sin parentesis
}
```

- ngx-translate es una librería de internacionalización para Angular. Para instalarla, debemos ejecutar los siguientes comandos en una terminal de sistema:

```
npm install @ngx-translate/core @ngx-translate/http-loader --save
```

Angular	@ngx-translate/core	@ngx-translate/http-loader
16+	15.x+	8.x+
13+ (ivy only)	14.x+	7.x+
10/11/12/13	13.x+	6.x+
9	12.x+	5.x+
8	12.x+	4.x+
7	11.x+	4.x+
6	10.x	3.x
5	8.x to 9.x	1.x to 2.x
4.3	7.x or less	1.x to 2.x
2 to 4.2.x	7.x or less	0.x

```
import { TranslateHttpLoader } from '@ngx-translate/http-loader';
```

```
@NgModule({  
  declarations: [  
    AppComponent  
  ],  
  imports: [  
    BrowserModule,  
    HttpClientModule,  
    TranslateModule.forRoot({  
      loader: {  
        provide: TranslateLoader,  
        useFactory: (http: HttpClient) => {  
          return new TranslateHttpLoader(http);  
        },  
        deps: [HttpClient]  
      }  
    })  
  ],  
  providers: []  
})
```

I18N

- El siguiente paso es la creación de los archivos de idioma correspondiente.

```
└─ assets
  └─ i18n
    {} de.json
    {} en.json
    {} es.json
    {} fr.json
```

```
{
  "texto.saludo": "Bienvenido"
}
```

```
{
  "texto.saludo": "Welcome"
}
```

- Diseñamos la vista:

```
<a class="nav-item nav-link" routerLink="#"
  (click)="cambiarIdioma('de')">
  
</a>
</ul>
</div>
</div>
</nav>

<br/>
{{ 'texto.saludo' | translate }}
```

```
export class AppComponent {  
  
    public idioma = 'es';  
  
    constructor(private translate: TranslateService){  
        this.translate.setDefaultLang(this.idioma);  
    }  
  
    public cambiarIdioma(idioma){  
        this.idioma = idioma;  
        this.translate.use(idioma);  
    }  
}
```

Que es Angular Material?

- [Angular Material](#) es una librería de estilos (*como Bootstrap*) basada en la guía de diseño de [Material Design](#), realizado por el equipo de Angular para integrarse perfectamente con Angular.

Instalación de Angular Material

```
ng add @angular/material
```

Componentes

- Layout
- Navegacion
- Form controls
- Botones
- Data table

Layout

- Card
- Divider
- Expansion Panel
- Grid list
- List
- Stepper
- Tabs
- Tree

Componentes

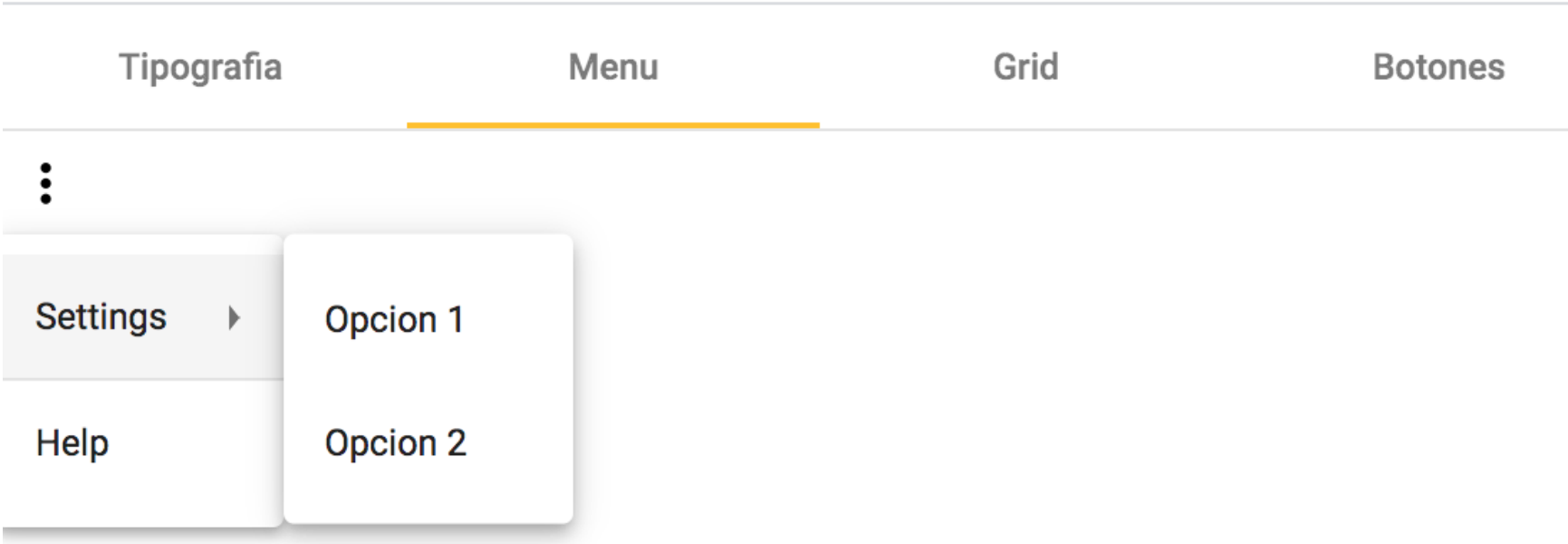
Tipografia	Menu	Grid	Botones
------------	------	------	---------

Navegación

- Menu
- Sidenav
- Toolbar

Navegación

- Ejemplo de Menus



Form Controls

- Autocomplete
- Checkbox
- Datepicker
- Form field
- Input
- Radio button
- Select
- Slider
- Slide toggle

Cajas de texto, campos input

Introduce nombre

Email:

Color:

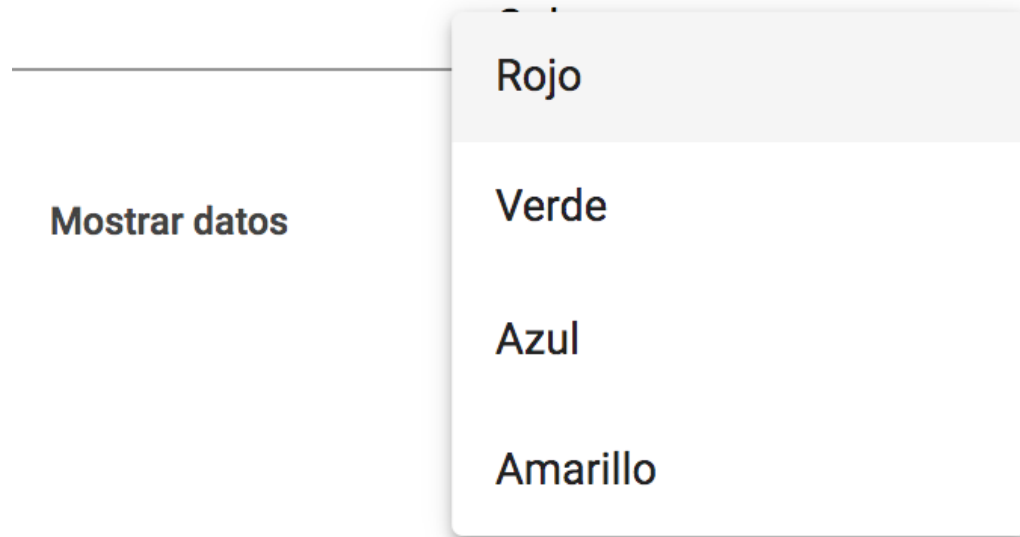
Checkbox, radio button y Slide toogle

☐ Aceptar condiciones

Select an option ☐ Option 1 ☐ Option 2

☐ Habilitar

Campos select, combos, listas desplegadas



Botones

- mat-button: Botón básico sin color de fondo
- mat-stroked-button: Botón con borde pero sin color de fondo
- mat-raised-button: Botón con sombra y con color de fondo
- mat-flat-button: Botón con color pero sin sombra
- mat-fab: Botón circular con fondo y sombra
- mat-mini-fab: Lo mismo que el anterior pero más pequeño
- mat-icon-button: Botón circular para añadir un icono, es transparente

<https://fonts.google.com/icons?hl=es-419>

```
<h2>Mini Fab</h2>
<button mat-mini-fab color="primary">Primary</button>
<button mat-mini-fab color="accent">Accent</button>
<button mat-mini-fab color="warn">Warn</button>
<button mat-mini-fab disabled>Disabled</button>
<button mat-mini-fab routerLink='.'>Link</button>
```

Botones

Button

Primary Accent Warn Disabled Link

Button stroked

Primary Accent Warn Disabled Link

Button raised

Primary Accent Warn Disabled Link

Button flat

Primary Accent Warn Disabled Link

Fab Button

Primary Accent Warn Disabled Link

Mini Fab

Primary Accent Warn Disabled Link

Icon Button



Data table

- Paginator
- Sort header
- Table

Data table

Marca	Precio	Ibu	Existencias
Alhambra verde	2.1	4	true
1906	2.00	3	true
Cruzcampo	1.5	2	true
Alhambra roja	2.3	1	true
Mahou	1.80	2	false
Corona	2.5	3	false
Estrella Galicia	1.8	3	true
Estrella Damm	2.2	1	false
Heineken	2	3	false
San Miguel	1.6	5	true

Tipografia

Name	CSS classes	Description
display-4	.mat-display-4	Large, one-off header, usually at the top of the page (e.g. a hero header).
display-3	.mat-display-3	Large, one-off header, usually at the top of the page (e.g. a hero header).
display-2	.mat-display-2	Large, one-off header, usually at the top of the page (e.g. a hero header).
display-1	.mat-display-1	Large, one-off header, usually at the top of the page (e.g. a hero header).
headline	.mat-h1, .mat-headline	Section heading corresponding to the <code><h1></code> tag.
title	.mat-h2, .mat-title	Section heading corresponding to the <code><h2></code> tag.
subheading-2	.mat-h3, .mat-subheading-2	Section heading corresponding to the <code><h3></code> tag.
subheading-1	.mat-h4, .mat-subheading-1	Section heading corresponding to the <code><h4></code> tag.
body-1	.mat-body, .mat-body-1	Base body text.
body-2	.mat-body-strong, .mat-body-2	Bolder body text.
caption	.mat-small, .mat-caption	Smaller body and hint text.
button	None. Used only in components.	Buttons and anchors.
input	None. Used only in components.	Form input fields.

Tipografia

```
<p class="mat-caption">Esto es un parrafo de ejemplo. mat-caption</p>
<p class="mat-h1">Esto es un parrafo de ejemplo. mat-h1</p>
<p class="mat-body">Esto es un parrafo de ejemplo. mat-body</p>
<p class="mat-body-strong">Esto es un parrafo de ejemplo. mat-body-strong</p>
<p class="mat-display-3">Esto es un parrafo de ejemplo. mat-display-3</p>
<p class="mat-display-2">Esto es un parrafo de ejemplo. mat-display-2</p>
<p class="mat-display-1">Esto es un parrafo de ejemplo. mat-display-1</p>
...
```


Estilos

- Los estilos personalizados se pueden sobrescribir en `styles.css`

Temas

- Temas de Angular Material:
 - `deeppurple-amber.css`
 - `indigo-pink.css`
 - `pink-bluegrey.css`
 - `purple-green.css`

Temas

- Temas personalizados:
- Para crear un tema personalizado agregamos los siguientes datos:
 - primary palette: colors most widely used across all screens and components.
 - accent palette: colors used for the floating action button and interactive elements.
 - warn palette: colors used to convey error state.
 - foreground palette: colors for text and icons.
 - background palette: colors used for element backgrounds.

Temas

- **Colores:**
- Paleta de Angular Material

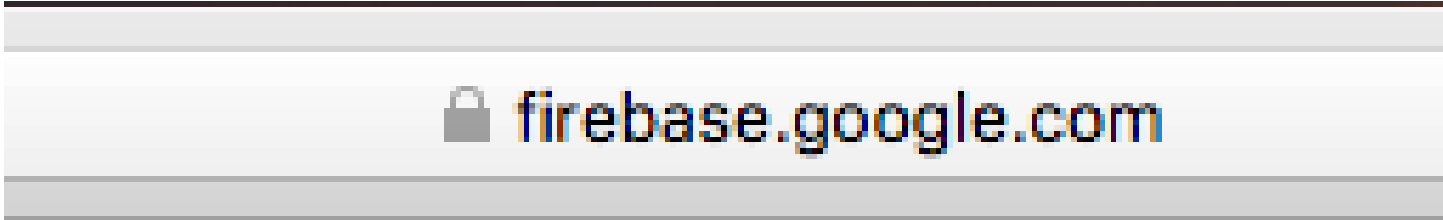
<https://material.io/design/color/the-color-system.html#tools-for-picking-colors>

- Paleta personalizada:

mcg.mbitson.com

Hosting

- Para desplegar la app utilizamos Firebase



 firebase.google.com

Firestore

- Vamos a la consola

IR A LA CONSOLA



- Creamos el proyecto:



Añadir proyecto

Crear el proyecto

- Introducimos el nombre del proyecto y pulsamos el botón continuar:



× Crear un proyecto(paso 1 de 3)

Comencemos con el
nombre de tu proyecto®

Nombre del proyecto

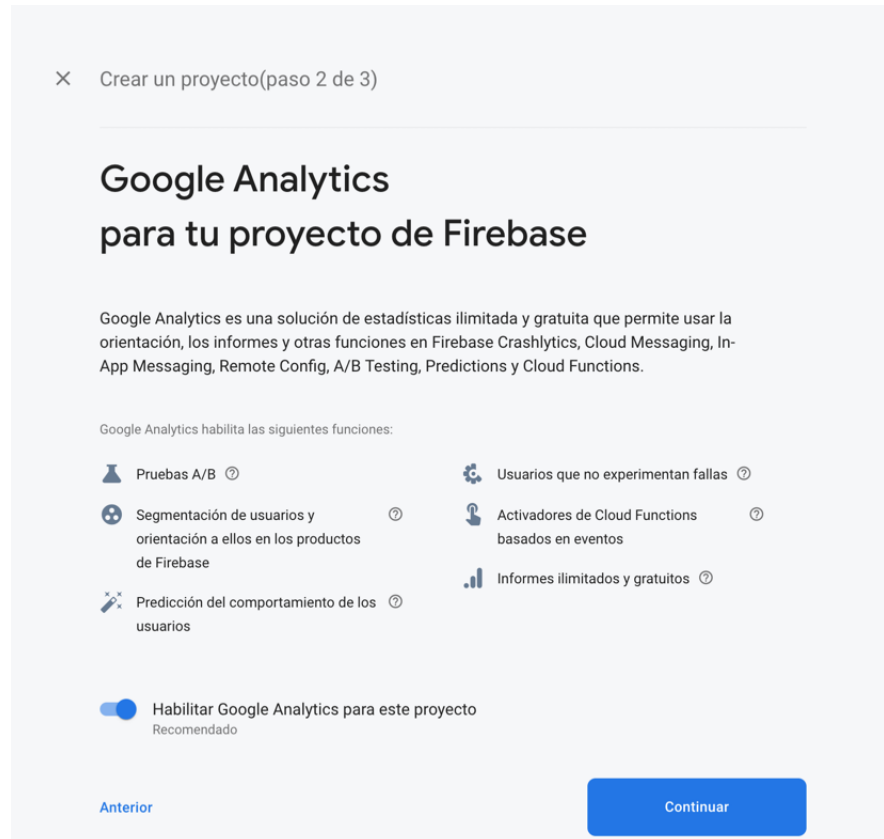
Prueba Deploy Anabel

prueba-deploy-anabel

Continuar

Crear el proyecto

- Deshabilitamos Google Analytics y pulsamos el botón continuar:



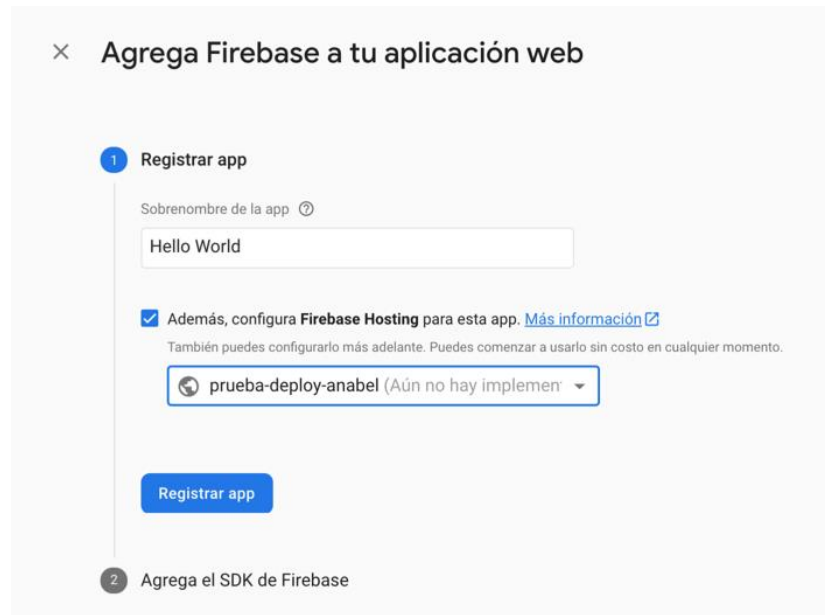
Crear el proyecto

- Una vez creado el proyecto agregamos la app web a nuestro proyecto:



Crear el proyecto

- Ponemos un nombre cualquiera y pulsamos el botón Registrar app:



The screenshot shows the 'Agregar Firebase a tu aplicación web' (Add Firebase to your web application) dialog in the Firebase console. It features a close button (X) in the top left corner. The dialog is divided into two steps: Step 1, 'Registrar app' (Register app), and Step 2, 'Agrega el SDK de Firebase' (Add the Firebase SDK). In Step 1, there is a text input field for 'Sobrenombre de la app' (App nickname) with a help icon, containing the text 'Hello World'. Below this is a checkbox labeled 'Además, configura Firebase Hosting para esta app' (Also, configure Firebase Hosting for this app), which is checked. A link for 'Más información' (More information) is provided. A note states: 'También puedes configurarlo más adelante. Puedes comenzar a usarlo sin costo en cualquier momento.' (You can also configure it later. You can start using it at no cost at any time.). Below the note is a dropdown menu for selecting a Firebase project, currently showing 'prueba-deploy-anabel' with a note '(Aún no hay implementaciones)' (No implementations yet). A blue 'Registrar app' button is at the bottom of Step 1. Step 2 is partially visible at the bottom.

× Agrega Firebase a tu aplicación web

1 Registrar app

Sobrenombre de la app ⓘ

Hello World

☒ Además, configura **Firebase Hosting** para esta app. [Más información](#) ⓘ

También puedes configurarlo más adelante. Puedes comenzar a usarlo sin costo en cualquier momento.

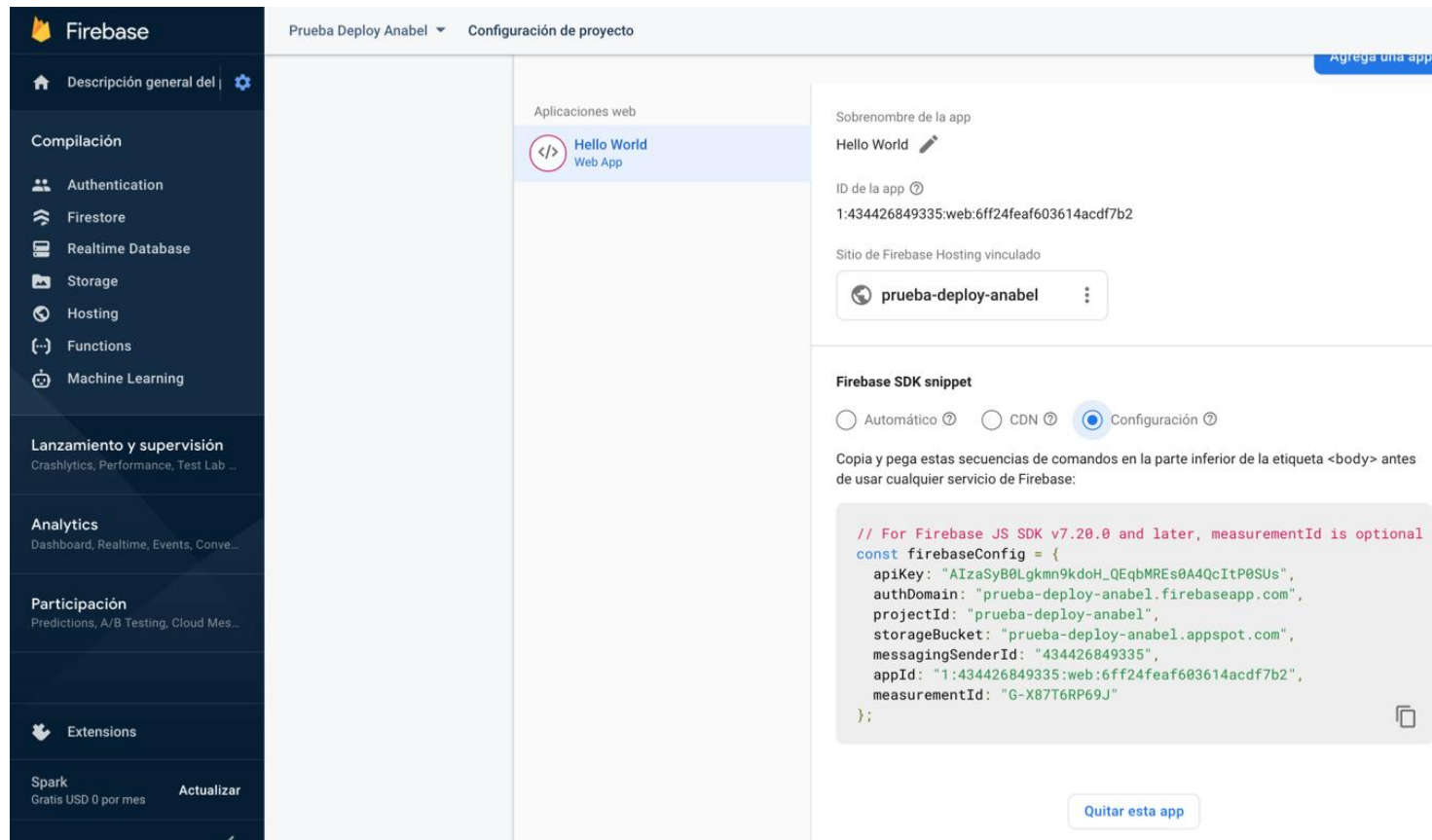
prueba-deploy-anabel (Aún no hay implementaciones)

Registrar app

2 Agrega el SDK de Firebase

Crear el proyecto

- Copiamos las credenciales del proyecto



The screenshot displays the Firebase console interface for a project named 'Prueba Deploy Anabel'. The left sidebar contains navigation links for various services: Descripción general del proyecto, Compilación, Authentication, Firestore, Realtime Database, Storage, Hosting, Functions, Machine Learning, Lanzamiento y supervisión, Analytics, Participación, Extensions, and Spark. The main content area is titled 'Configuración de proyecto' and shows a list of web applications under 'Aplicaciones web'. The 'Hello World' web app is selected, and its configuration details are displayed on the right. The configuration includes the app's display name 'Hello World', its ID '1:434426849335:web:6ff24feaf603614acdf7b2', and the linked Firebase Hosting site 'prueba-deploy-anabel'. Below this, the 'Firebase SDK snippet' section shows three options: 'Automático', 'CDN', and 'Configuración' (which is selected). A text box provides instructions to copy and paste the SDK snippet into the HTML body of the app. The snippet is a JavaScript code block that initializes the Firebase SDK with specific credentials. A 'Quitar esta app' button is located at the bottom right of the configuration panel.

Prueba Deploy Anabel Configuración de proyecto

Agrega una app

Aplicaciones web

Hello World Web App

Sobrenombre de la app

Hello World

ID de la app

1:434426849335:web:6ff24feaf603614acdf7b2

Sitio de Firebase Hosting vinculado

prueba-deploy-anabel

Firebase SDK snippet

☐ Automático ☐ CDN ☒ Configuración

Copia y pega estas secuencias de comandos en la parte inferior de la etiqueta <body> antes de usar cualquier servicio de Firebase:

```
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyB0Lgkmn9kdoH_QEqbMRs0A4QcItP8SUs",
  authDomain: "prueba-deploy-anabel.firebaseio.com",
  projectId: "prueba-deploy-anabel",
  storageBucket: "prueba-deploy-anabel.appspot.com",
  messagingSenderId: "434426849335",
  appId: "1:434426849335:web:6ff24feaf603614acdf7b2",
  measurementId: "G-X87T6RP69J"
};
```

Quitar esta app

Credenciales en Angular

- Copiamos las credenciales en los archivos environment.ts y environment.prod.ts:

```
environment.ts — Angular
TS environment.ts X
ejemplo1-hello-world > src > environments > TS environment.ts > ...
1  // This file can be replaced during build by using the `file
2  // `ng build --prod` replaces `environment.ts` with `environ
3  // The list of file replacements can be found in `angular.js
4
5  export const environment = {
6    production: false,
7    firebaseConfig: {
8      apiKey: "AIzaSyB0Lgkmn9kdoH_QEqbMREs0A4QcItP0SUs",
9      authDomain: "prueba-deploy-anabel.firebaseio.com",
10     projectId: "prueba-deploy-anabel",
11     storageBucket: "prueba-deploy-anabel.appspot.com",
12     messagingSenderId: "434426849335",
13     appId: "1:434426849335:web:6ff24feaf603614acdf7b2",
14     measurementId: "G-X87T6RP69J"
15   }
16 };
17
18
```

```
environment.prod.ts — Angular
TS environment.ts TS environment.prod.ts X
ejemplo1-hello-world > src > environments > TS environment.prod.ts > [e] envirom
1  export const environment = {
2    production: true,
3    firebaseConfig: {
4      apiKey: "AIzaSyB0Lgkmn9kdoH_QEqbMREs0A4QcItP0SUs",
5      authDomain: "prueba-deploy-anabel.firebaseio.com",
6      projectId: "prueba-deploy-anabel",
7      storageBucket: "prueba-deploy-anabel.appspot.com",
8      messagingSenderId: "434426849335",
9      appId: "1:434426849335:web:6ff24feaf603614acdf7b2",
10     measurementId: "G-X87T6RP69J"
11   }
12 };
13
```

Instalamos las herramientas de firebase

- Instalamos las herramientas de firebase desde la consola,
- Debemos estar situados dentro del proyecto:

```
npm install firebase-tools
```

Login

- En la consola, nos logamos con la misma cuenta de Gmail que utilizamos para acceder a la consola de Firebase:

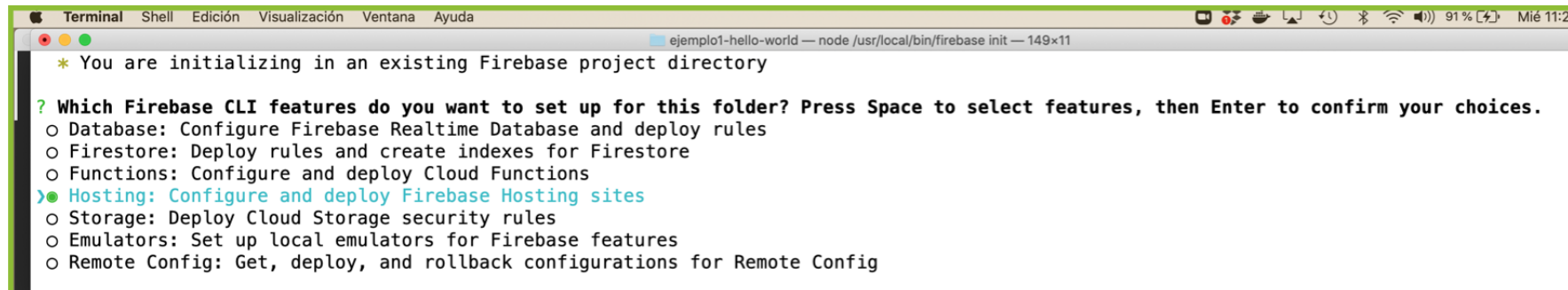
```
firebase login
```

Init

```
firebase init
```

Init

- Me pide que seleccione que quiero hacer con el proyecto: Hosting
- Solo esto aunque utilicemos Authentication y BBDD Firestore.



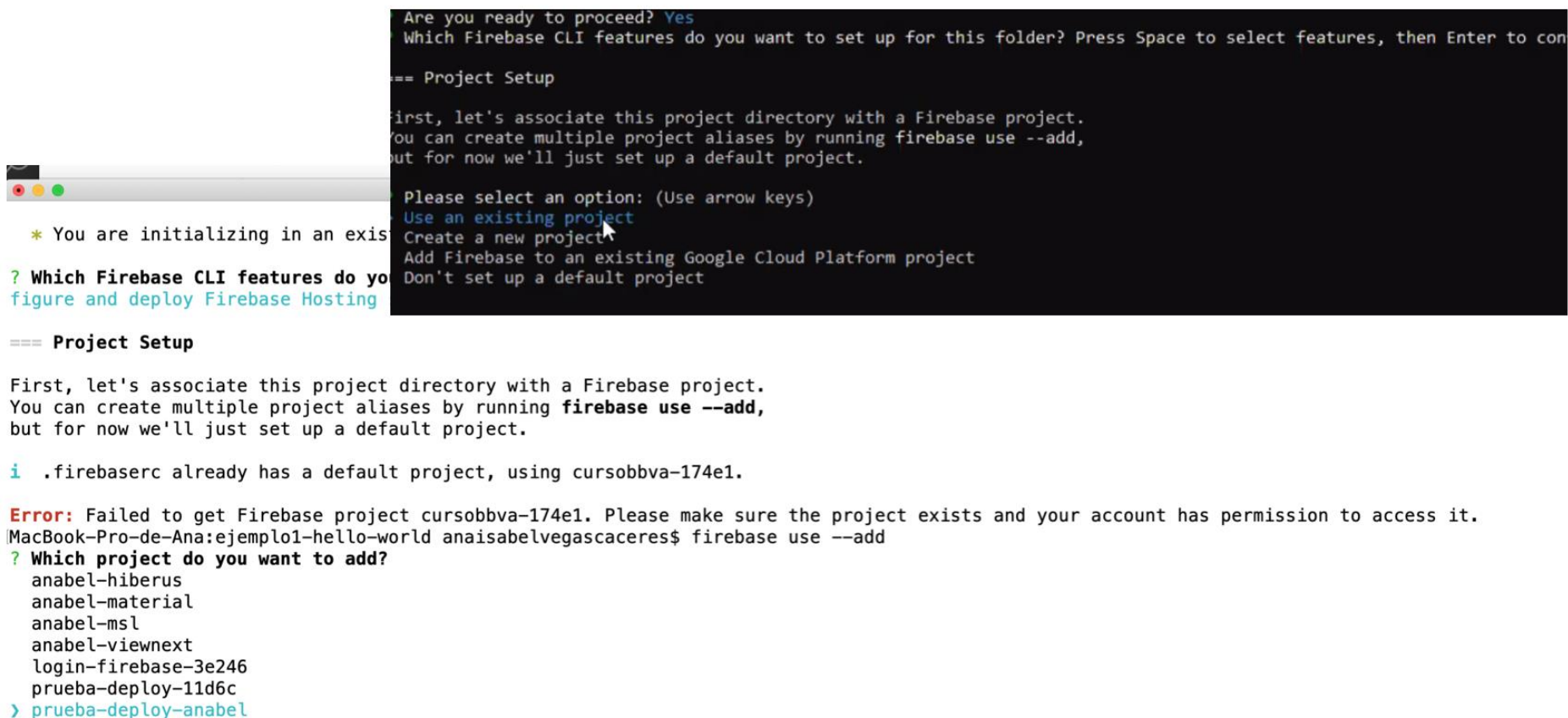
```
Terminal Shell Edición Visualización Ventana Ayuda
ejemplo1-hello-world — node /usr/local/bin/firebase init — 149x11

* You are initializing in an existing Firebase project directory

? Which Firebase CLI features do you want to set up for this folder? Press Space to select features, then Enter to confirm your choices.
  ○ Database: Configure Firebase Realtime Database and deploy rules
  ○ Firestore: Deploy rules and create indexes for Firestore
  ○ Functions: Configure and deploy Cloud Functions
  ● Hosting: Configure and deploy Firebase Hosting sites
  ○ Storage: Deploy Cloud Storage security rules
  ○ Emulators: Set up local emulators for Firebase features
  ○ Remote Config: Get, deploy, and rollback configurations for Remote Config
```


Init

- Selecciono un proyecto existente y elijo:



```
Are you ready to proceed? Yes
Which Firebase CLI features do you want to set up for this folder? Press Space to select features, then Enter to con

=== Project Setup

First, let's associate this project directory with a Firebase project.
You can create multiple project aliases by running firebase use --add,
but for now we'll just set up a default project.

Please select an option: (Use arrow keys)
- Use an existing project
- Create a new project
- Add Firebase to an existing Google Cloud Platform project
- Don't set up a default project

* You are initializing in an exist
? Which Firebase CLI features do yo
figure and deploy Firebase Hosting

=== Project Setup

First, let's associate this project directory with a Firebase project.
You can create multiple project aliases by running firebase use --add,
but for now we'll just set up a default project.

i .firebaseerc already has a default project, using cursobbva-174e1.

Error: Failed to get Firebase project cursobbva-174e1. Please make sure the project exists and your account has permission to access it.
MacBook-Pro-de-Ana:ejemplo1-hello-world anaisabelvegascaceres$ firebase use --add
? Which project do you want to add?
  anabel-hiberus
  anabel-material
  anabel-msl
  anabel-viewnext
  login-firebase-3e246
  prueba-deploy-11d6c
> prueba-deploy-anabel
```

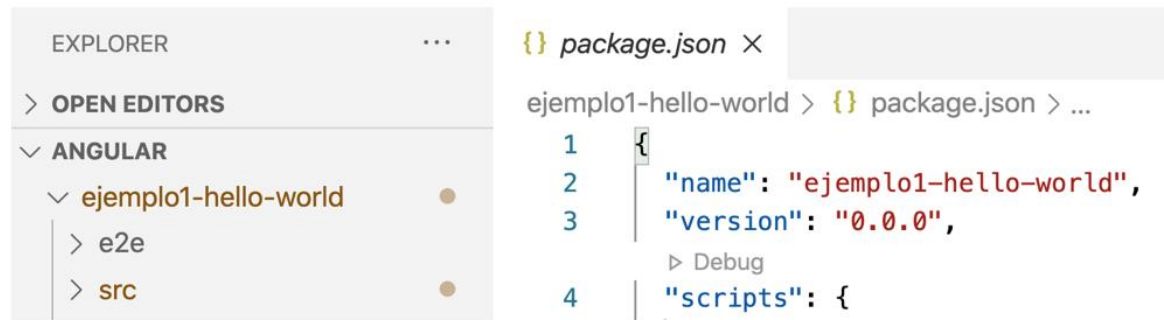
Init

- Si tengo configurado un proyecto por defecto, lo puedo cambiar de la siguiente forma:

```
cd ~/firebase-database-instance1  
MacBook-Pro-de-Ana:ejemplo1-hello-world anaisabelvegascaceres$ firebase use --add  
? Which project do you want to add? prueba-deploy-11d6c  
? What alias do you want to use for this project? (e.g. staging) default
```

Init

- El directorio publico es dist/nombre_proyecto:
- El nombre del proyecto lo vemos en package.json



```
What do you want to use as your public directory? dist/ejemplo1-hello-world  
Configure as a single-page app (rewrite all urls to /index.html)? (y/N) y_
```

Init

- SPA? Y

```
? Configure as a single-page app (rewrite all urls to /index.html)?  
(y/N) y
```

Init

- Set up automatic builds and deploys with GitHub? No

```
? What do you want to use as your public directory? dist/ejemplo1-hello-world  
? Configure as a single-page app (rewrite all urls to /index.html)? Yes  
? Set up automatic builds and deploys with GitHub? No
```

Empaquetar

- Creamos la versión de producción:

```
ejemplo1-hello-world>ng build
```

Deploy

```
firebase deploy
```

- Nos muestra la url para poder acceder

```
Project Console: https://console.firebase.google.com/pro  
Hosting URL: https://prueba-deploy-carlos.web.app
```

Angular y WS REST



Completa nuestra encuesta
de satisfacción a través del QR



GRACIAS

indra